

내비게이션

타이완에는 3,000미터 이상 높이의 봉우리가 250개 있을 정도로 산이 많다. 아밍은 타이완의 고산지대에 멀리 떨어져 있는 마을끼리 물건을 주고받을 수 있는 네트워크를 만들려고 한다.

아밍의 계획에서 각 네트워크는 0부터 $N - 1$ 까지 번호가 매겨진 N 개의 노드가 있다. 이 노드 중에서 정확히 $K = 6$ 개는 충전소이고 나머지 $N - k$ 개의 노드는 마을을 나타낸다. 이 노드들은 0부터 $N - 2$ 까지 번호가 매겨진 $N - 1$ 개의 양방향 케이블로 연결되어 있다. 각 i 에 대해 ($0 \leq i \leq N - 2$), 케이블 i 는 노드 $U[i]$ 와 노드 $V[i]$ 를 연결한다. 각 케이블은 서로 다른 두 노드를 연결하며, 한 쌍의 노드는 최대 한개의 케이블로 연결된다. 어떤 두 노드를 고르더라도 케이블을 통해 연결됨이 보장된다.

노드 a 와 노드 b 의 거리는 $d(a, b)$ 로 표현하며 다음과 같이 정의된다.

- 만약 $a = b$ 이면 $d(a, b) = 0$.
- 그렇지 않으면, $d(a, b)$ 는 a 에서 b 까지 이동하는데 필요한 케이블의 최소 수

이 네트워크의 분기 지수가 최소 δ 이상이라는 뜻은, 네트워크의 각각의 노드가 정확히 한 개의 케이블에 연결되어 있거나, 또는 적어도 δ 개의 케이블에 연결되어 있다는 것이다. 아밍은 이 네트워크의 분기 지수가 최소 양의 정수 B 라는 것을 알고 있다.

아밍은 0, 1, ..., 99로 번호가 매겨진 100가지 서로 다른 타입의 케이블을 준비했다. 네트워크의 케이블 각각은 이 중 한 타입이다.

배달 임무를 수행하기 위해서 로봇은 케이블 네트워크를 타고 이동할 것이다. 아밍은 로봇이 충전소로 돌아와서 스스로 충전할 수 있도록 내비게이션 프로그램을 설치하려고 한다. 그러나, 로봇에 설치된 메모리가 매우 작기 때문에, 내비게이션 프로그램을 개발할 때 로봇이 이용할 수 있는 정보는 충전소의 수 $K = 6$, 보장된 분기 지수 하한 B , 그리고 로봇의 현재 위치에 연결된 케이블의 타입 뿐이다.

로봇이 둘 이상의 케이블이 연결된 마을 노드 u 에서 경로를 찾으려면, 로봇은 먼저 u 에 연결된 케이블들을 임의의 순서로 스캔한다. 이 순서대로 케이블의 종류 정보가 내비게이션 프로그램에 전달된다. 각 케이블마다, 프로그램은 이 케이블을 타고 이동했을 때 거리가 줄어드는 충전소들의 수를 세야 한다.

구체적으로, $v_1, \dots, v_k$ ($k \geq 2$) 가 u 에 케이블로 직접 연결된 노드들이라고 하고, c_i ($1 \leq i \leq k$)가 노드 u 와 노드 v_i 를 잇는 케이블의 타입이라고 하자. 프로그램에는 K , B , 그리고 리스트 $c_1, c_2, \dots, c_k$ 가 주어진다. 각 i 에 대해서 ($1 \leq i \leq k$), 프로그램은 $d(v_i, p) < d(u, p)$ 가 되는 충전소 p 의 수를 알아내야 한다.

당신이 할 일은 케이블의 타입을 할당하는 전략을 설계하고, 이를 이용한 내비게이션 프로그램을 설계하는 것이다. 당신의 점수는 당신의 해법이 사용하는 케이블의 종류 수에 의해 결정된다. (Subtasks, Scoring에서 자세한 내용을 보라)

Implementation details

두 함수를 구현해야 하는데, 하나는 케이블의 타입을 할당하는 것이고 다른 하나는 로봇의 프로그램이다.

케이블의 타입을 할당하는 함수는 다음과 같다.

```
std::vector<int> construct_network(int N, int K, int B,
                                   std::vector<int> U, std::vector<int> V,
                                   std::vector<int> P)
```

- N : 노드의 수
- K : 충전소의 수
- B : 네트워크의 최소 분기 지수 (이 값은 보장된다)
- U, V : 케이블 연결을 설명하는 길이 $N - 1$ 인 배열
- P : 충전소 노드를 나타내는 길이 $K = 6$ 인 배열
- 이 함수는 각 테스트 케이스에 대해 **최대 10 000번** 호출될 수 있다.

이 함수는 길이 $N - 1$ 인 배열 T 을 리턴해야 한다.

- $T[i]$ 는 노드 $U[i]$ 와 $V[i]$ 를 연결하는 케이블의 타입.
- 각각 $T[i]$ 는 $0 \leq T[i] < 100$ 여야 한다.

로봇의 프로그램을 구현하는 함수는 다음과 같다.

```
std::vector<int> navigate(int K, int B, std::vector<int> C)
```

- K : 충전소의 수
- B : 네트워크의 최소 분기 지수 (이 값은 보장된다)
- C : 2개 이상의 케이블에 연결된 어떤 마을 노드에 연결된 모든 케이블의 타입의 리스트 (임의의 순서)
- C 의 길이가 최소 B 임은 보장된다.
- 이 함수는 각 테스트 케이스에 대해 **최대 100 000번** 호출될 수 있다.
- `navigate` 함수는 원래 네트워크 구조에 의존하면 안된다. 구체적으로, 채점 시스템은 하나의 테스트 케이스 내에서도 구성된 서로 다른 네트워크들에 대한 `navigate` 호출들을 임의의 순서로 재배열할 수 있다.

이 함수는 배열 D 를 리턴해야 한다.

- D 의 길이는 C 와 같아야 한다.
- $D[i]$ 는 $C[i]$ 로 주어진 케이블을 따라가면 거리가 줄어드는 충전소의 수이다.

채점 시스템에서 `construct_network`와 `navigate` 함수는 **두 별도의 프로세스**에서 호출되는데 유의하라.

Constraints

- $K = 6$.
- $K < N \leq 100\,000$.
- 각 테스트 케이스에서 모든 `construct_network` 호출에 대한 N 값의 총합은 100 000 이하이다.
- 각 $0 \leq i \leq N - 2$ 에 대해 $0 \leq U[i] < V[i] < N$.
- 어떤 두 노드를 고르더라도 케이블을 통해 연결된다.
- $0 \leq P[0] < P[1] < \dots < P[K - 1] < N$.
- 각 배열 C 는 최소 2개의 케이블에 연결된 어떤 마을 노드에 연결된 모든 케이블 타입의 순열이다.
- 각 테스트 케이스에서 모든 `navigate` 호출에 대한 배열 C 길이의 총합은 200 000 이하이다.

Subtasks and Scoring

서브태스크	점수	추가적인 제약조건
1	6	$B = 2, N = 7$.
2	14	$B = 2, 0 \leq i < N - 1$ 인 각 i 에 대해 $U[i] = i, V[i] = i + 1$.
3	20	$B = 5$.
4	20	$B = 4$.
5	40	$B = 2$.

어떤 테스트 케이스에서도, 만약 다음 중 한 가지 조건에 걸리면 이 테스트 케이스에 대한 점수는 0이다. (CMS에서 Output isn't correct)

- `construct_network`의 리턴값이 타당하지 않다.
- `navigate`의 리턴값이 정확하지 않다.

그렇지 않다면, S 가 각각 `construct_network` 호출이 리턴한 배열 T 의 성분 중 가장 큰 값+1이라고 하자. 다르게 설명하면, 모든 `construct_network`이 만든 네트워크는 케이블 타입 $0, 1, \dots, S - 1$ 만 써서 만들 수 있는 S 중 최소값이다.

각 서브태스크의 점수는 다음과 같이 S 에 의해 결정된다.

조건	Subtask 1	Subtask 2	Subtask 3 and 4	Subtask 5
$100 < S$	0	0	0	0
$16 \leq S \leq 100$	2	2	$12 - \log_2 S$	$24 - 2 \log_2 S$
$10 \leq S \leq 15$	2	4	$16 - 0.5S$	$32 - S$
$7 \leq S \leq 9$	2	6	$21 - S$	$42 - 2S$
$S = 6$	2	10	15	30
$S = 5$	6	14	17.5	40
$S \leq 4$	6	14	20	40

특히 Subtask 1, 2, 5에서 $S \leq 5$ 이면 만점을 받고, Subtask 3, 4에서는 $S \leq 4$ 이면 만점을 받는다.

Example

$N = 9, K = 6, B = 2$ 인 다음 시나리오를 생각해보자.

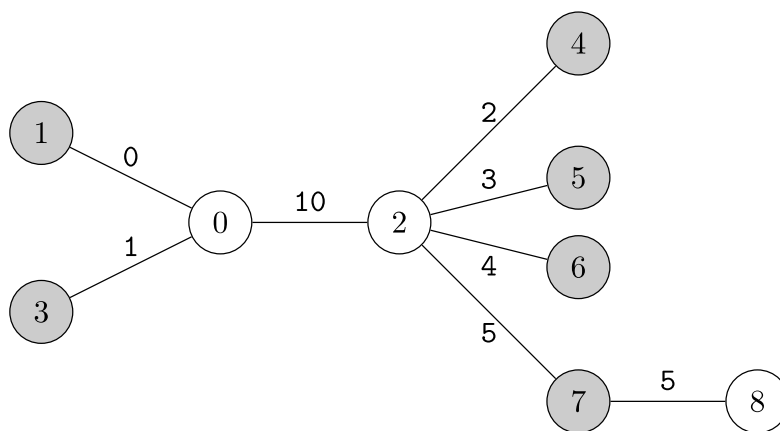
이 네트워크는 구체적으로 $U = [0, 0, 0, 2, 2, 2, 2, 7], V = [1, 2, 3, 4, 5, 6, 7, 8]$ 로 주어진다. 충전소는 $P = [1, 3, 4, 5, 6, 7]$ 이다. 첫번째 프로세스에 대해 다음 호출을 생각해보자.

```
construct_network(9, 6, 2, [0, 0, 0, 2, 2, 2, 2, 7],
                  [1, 2, 3, 4, 5, 6, 7, 8],
                  [1, 3, 4, 5, 6, 7])
```

아밍이 다음과 같이 케이블 타입을 지정해줬다고 하자.

```
[0, 10, 1, 2, 3, 4, 5, 5]
```

결과로 만들어지는 네트워크는 다음과 같다.



두번째 프로세스에서 로봇이 노드 0에서 출발한다고 하자. 노드 0은 노드 1,2,3과 연결되어 있다. 만약 로봇이 이 순서대로 케이블을 읽는다고 하면 다음과 같이 호출할 것이다.

```
navigate(6, 2, [0, 10, 1])
```

네트워크 구조에 의하면 다음과 같다.

- 노드 1로 가면 이 노드가 충전소여서 거리가 짧아진다. 구체적으로는, $0 = d(1,1) < 1 = d(0,1) < 2 = d(2,1) = d(3,1)$.
- 노드 3으로 가면 이 노드가 충전소여서 거리가 짧아진다. 구체적으로는, $0 = d(3,3) < 1 = d(0,3) < 2 = d(1,3) = d(2,3)$.
- 다른 충전소인 노드 4, 5, 6, 7은 노드 2가 더 가깝다. 구체적으로는, $u = 4,5,6,7$ 일 때 $1 = d(2,u) < 2 = d(0,u) < 3 = d(1,u) = d(3,u)$.

따라서 케이블 (0,1), (0,2), (0,3)을 따라 이동했을 때 거리가 가까워지는 충전소의 수는 각각 1, 4, 1이다. 따라서 리턴값은 다음과 같아야 한다.

```
[1, 4, 1]
```

`navigate` 함수가 C 의 다른 순열을 가지고 호출될 수 있음에 유의하라. 예를 들어, `navigate(6, 2, [1, 0, 10])`도 노드 0에 대한 타당한 호출이다. 이 경우 함수의 리턴값은 $[1, 1, 4]$ 이어야 한다.

로봇이 노드 2에서 출발한다고 하자. 다음과 같은 호출을 하게 되고

```
navigate(6, 2, [10, 2, 3, 4, 5])
```

리턴값은 다음과 같아야 한다.

```
[2, 1, 1, 1, 1]
```

이 네트워크에서, 'navigate'는 다른 노드에서 호출될 수 없는데

- 노드 1,3,4,5,6,7는 모두 충전소이고,
- 노드 8은 하나의 케이블로만 연결되어 있기 때문이다.

이 테스트 케이스에서 사용한 케이블의 타입들은 0,1,2,3,4,5,10이다. 따라서, $S = 11$ 이 스코어를 정하는데 쓰인다.

이 예제에 추가로, 이 문제에 첨부된 패키지에는 샘플 그레이더를 위한 다른 두 예제 입출력이 포함되어 있다.

Sample Grader

샘플 그레이더는 같은 프로세스에서 `construct_network`와 `navigate`를 호출한다. 추가로, 샘플 그레이더가 `navigate`를 호출할 때, 입력에서 주어진 순서대로 케이블 타입이 주어진다.

위의 두 가지 성질은 실제 그레이더와 다르다.

입력 양식:

```
T
(scenario 0)
(scenario 1)
...
(scenario T-1)
```

각 시나리오에는 다음과 같은 양식으로 주어진다.

```
N
B
U[0] V[0]
U[1] V[1]
...
U[N-2] V[N-2]
K
P[0] P[1] ... P[K-1]
Q
X[0] X[1] ... X[Q-1]
```

각 시나리오에서, 샘플 그레이더는 `navigate`를 Q 번 호출할 것이며, i 번째 호출에서는 노드 $X[i - 1]$ 의 정보를 이용한다. 샘플 그레이더는 K 값이 6이 아닌 다른 값을 가지는 것을 지원함에 유의하라.

출력 양식:

만약 `construct_network`나 `navigate`가 리턴한 배열이 타당하지 않다면, 샘플 그레이더는 비정상적으로 종료하고 이유를 출력한다. 그렇지 않다면, 샘플 그레이더는 `navigate`가 리턴한 배열 D 를 한 줄에 출력한다.

모든 시나리오를 처리한 후, 샘플 그레이더는 표준 에러에 한 줄을 출력한다.

```
S = S'
```

S' 는 테스트 케이스에서 `construct_network`이 리턴한 모든 배열 T 에 포함된 값들의 최대값+1이다.